Submission - E1 Kafka - Combined Test Results

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Assignment: Exercise 1 - Kafka Getting Started
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Quick Access Links

- Repository: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests
- Main branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/main
- Marco branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/marco
- Roman branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/roman
- Evan branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/evan

Contribution Mapping

- Marco Birchler: Experiment design and execution on `marco`
- Roman Babukh: Experiment framework and outputs on `roman`
- Evan Martino: Producer-focused experiment runs and review on `evan`

Scope and Method

This document is the combined, high-level E1 report on `main`. It consolidates results from:

- `marco` branch
- `roman` branch
- `evan` branch

The structure follows the E1 task categories:

1. Producer experiments
2. Consumer experiments
3. Fault tolerance and reliability

Branch-Based Evidence Map

- `marco`: `TEST_REPORT.md`, `test-configs/`, `Stats/`, `Logs/`
- `roman`: `results/review.md`, `results/producer/summary.md`, `results/consumer/summary.md`, `results/fault_tolerance/summary.md`
- `evan`: `review.md`, `plots/`

---

1) Producer Experiments

1.1 Batch Size and Latency

Observed trend across the branches: batching materially affects throughput/latency characteristics, but the absolute impact depends on the benchmark harness.

- Marco - branch `marco`: small and large batch setups are both around `~145 msg/s`, with application-side sleep identified as the dominant bottleneck.
- Roman - branch `roman`: `small_batch` = `9195.66 msg/s` (p95 `2095.639 ms`) vs `large_batch` = `161763.76 msg/s` (p95 `85.241 ms`).
- Evan - branch `evan`: larger batch sizes increased latency but did not significantly improve throughput in the measured single-broker setup.

Interpretation:

- All branches support the same conceptual result from Kafka behavior: larger batches reduce request overhead and improve effective broker utilization.
- The much stronger delta on `roman` indicates that client-side pacing and benchmark harness design can dominate measured Kafka tuning effects.
- Evan's result reinforces the environment-dependency point: under tighter local bottlenecks, batching effects can be visible in latency before they appear in throughput.
- Practical takeaway: optimize producer-side rate limiting and message generation path before concluding that a Kafka config does not matter.

Source citations:

- Marco: `origin/marco:TEST_REPORT.md`
- Roman: `origin/roman:results/review.md`
- Evan: `origin/evan:review.md`

1.2 Acknowledgment Settings (`acks`)

- Marco - branch `marco`: throughput remains similar for `acks=0`, `acks=1`, and `acks=all` (`~143–150 msg/s`) under low-rate local conditions.
- Roman - branch `roman`: clear ordering by performance and latency:
  - `acks=0`: `371761.9 msg/s`, p95 `8.815 ms`
  - `acks=1`: `95213.9 msg/s`, p95 `151.516 ms`
  - `acks=all`: `113533.63 msg/s`, p95 `139.867 ms`
- Evan - branch `evan`: `acks=0` showed the highest throughput, while `acks=1` and `acks=all` reduced throughput with only a small gap between `acks=1` and `acks=all` in the single-broker setup.

Interpretation:

- The reliability/performance trade-off exists in both branches, but it is statistically clearer in higher-throughput and less throttled runs.
- Marco results show that under moderate load, `acks=all` can be selected with little observed penalty.
- Roman results show that `acks=0` can massively improve raw speed, but this setting removes durability guarantees and should not be used for reliability-critical paths.
- Evan's findings align with the same trade-off and add consistency for local single-broker conditions.
- Combined policy implication: default to `acks=all`, and only relax when message loss is acceptable by design.

Source citations:

- Marco: `origin/marco:TEST_REPORT.md`
- Roman: `origin/roman:results/review.md`
- Evan: `origin/evan:review.md`

1.3 Brokers, Partitions, and Load Scaling

- Marco - branch `marco`: increasing partitions from 1 to 4 raises broker CPU from `~50%` to `~75%`, while single-producer throughput remains around `~150 msg/s`.
- Marco - branch `marco` load test: scaling from 1 to 3 producers increases aggregate throughput from `~145` to `~514 msg/s`, with CPU near saturation at peak.
- Roman - branch `roman`: higher-end automated producer numbers show that benchmark setup can expose substantially larger throughput envelopes.
- Evan - branch `evan`: producer scaling increased throughput, but increasing partitions alone did not significantly improve throughput on the single-broker laptop environment.

Interpretation:

- Partition increases are not free: coordination and bookkeeping overhead rises even when throughput does not immediately improve.
- Horizontal producer scaling clearly works and is required to expose broker limits in this project environment.
- The two branches together show a consistent pattern: single-client tests can understate Kafka capacity, while multi-client tests reveal scaling behavior.
- Evan's partition-scaling observation adds a practical boundary condition: partition count gains can remain limited when all partitions share the same broker hardware.
- Capacity planning implication: evaluate partition count and producer parallelism together, not in isolation.

Source citations:

- Marco: `origin/marco:TEST_REPORT.md`
- Roman: `origin/roman:results/review.md`
- Evan: `origin/evan:review.md`

---

2) Consumer Experiments

2.1 Consumer Lag Under Processing Delay

- Marco - branch `marco`: normal consumption at `~255 msg/s`; with artificial delay, lag grows heavily (`24k` produced vs `1.7k` consumed in one scenario).
- Roman - branch `roman`: delay comparison on 1500 messages:
  - `delay_0ms`: consumed `1500/1500`, max lag `0`, throughput `355.57 msg/s`
  - `delay_10ms`: consumed `43/1500`, max lag `1475`, end lag `1457`, throughput `1.33 msg/s`

Interpretation:

- Both branches strongly confirm that small processing delays can collapse effective throughput and create sustained lag.
- Once lag growth starts, downstream effects include delayed analytics, retention-pressure risk, and possible rebalance instability.
- The close agreement between branches increases confidence that this is a fundamental consumer behavior, not an artifact of one implementation.
- Operational implication: monitor lag as a first-class SLO signal and keep per-message processing paths minimal.

Source citations:

- Marco: `origin/marco:TEST_REPORT.md`
- Roman: `origin/roman:results/review.md`

2.2 Offset Reset Misconfiguration Risk

- Marco - branch `marco`: with `latest`, approximately `63%` effective data loss in scenarios where production starts before consumer.
- Roman - branch `roman`:
  - `earliest`: old `99/100`, new `25/25`
  - `latest`: old `0/100`, new `25/25`

Interpretation:

- Both branches reproduce the same critical risk: `latest` on a fresh group skips already written records.
- This is technically expected behavior, but in practice it is often perceived as accidental data loss when consumers start late.
- The consistency of findings across branches makes this one of the strongest configuration lessons from E1.
- Default recommendation for analysis/backfill scenarios: use `earliest` unless there is a strict reason not to.

Source citations:

- Marco: `origin/marco:TEST_REPORT.md`
- Roman: `origin/roman:results/review.md`

2.3 Multiple Consumers and Group Behavior

- Marco - branch `marco`: same-group consumers split partitions approximately `50/50`; different groups each receive full stream copies (pub-sub behavior).
- Roman - branch `roman`: poll-interval experiment shows group stability is sensitive to processing duration:
  - `low_poll_interval`: `max_poll_exceeded=True`, consumed `1/8`
  - `high_poll_interval`: consumed `8/8`

Interpretation:

- The combined evidence links two core dimensions of consumer correctness: assignment semantics (group topology) and liveness semantics (poll timing).
- Correct partition distribution alone is insufficient if poll interval constraints are violated under processing load.
- Configuration implication: tune `max.poll.interval.ms` to real processing time and validate behavior under stress, not only at nominal load.
- Architectural implication: if processing can exceed poll limits, move heavy logic out of the poll thread or decouple via worker pipelines.

Source citations:

- Marco: `origin/marco:TEST_REPORT.md`
- Roman: `origin/roman:results/review.md`

---

3) Fault Tolerance and Reliability

3.1 Broker Failure and Leader Election

- Marco - branch `marco`:
  - resilient config (`acks=all`, retries enabled): producer continued (`195` msgs sent)
  - fragile config (`acks=1`, retries disabled): early failure (`93` msgs sent)
- Roman - branch `roman`:
  - failover time `19.714 s`
  - producer sent `9001`, delivered `9001`, errors `0`
  - consumer processed `8049`
  - producer p95 latency spike `19176.295 ms`

Interpretation:

- Both branches show the same fault pattern: failover is survivable but not transparent in latency.
- Client settings are decisive: retries and stronger durability semantics can bridge broker transitions, while weaker settings fail faster.
- Roman's near-20s election window quantifies the size of the expected disruption interval during leadership change.
- Reliability implication: evaluate failover behavior with latency objectives, not just delivered-message counts.

Source citations:

- Marco: `origin/marco:TEST_REPORT.md`
- Roman: `origin/roman:results/review.md`

3.2 Durability and Message Loss Risk

- Marco - branch `marco`:
  - `RF=1`: `67.8%` loss in failure scenario
  - high-durability setup (`RF=2`, strict ISR): `19.7%` failed-write/loss effect under degraded cluster
- Roman - branch `roman`:

    - replicated setup with `acks=all` and controlled failover maintained full producer delivery in the measured run

Interpretation:

- `RF=1` shows an unacceptable loss profile for any reliability-sensitive workload.
- Replication plus stronger acknowledgment semantics materially improves resilience, but may reject writes when ISR constraints cannot be met.
- Cross-branch conclusion: durability controls shift failure mode from silent loss toward explicit backpressure/rejection, which is operationally preferable.
- Design implication: choose replication and ISR policy according to business tolerance for temporary unavailability versus permanent data loss.

Source citations:

- Marco: `origin/marco:TEST_REPORT.md`
- Roman: `origin/roman:results/review.md`

Consolidated Conclusions

1. Keep `acks=all` for reliability-sensitive use cases; benchmark cost in your own environment.
2. Avoid `RF=1` for critical data paths.
3. Monitor consumer lag continuously; lag growth is an early warning for instability.
4. Treat `auto.offset.reset` as a data-safety setting, not a convenience default.
5. Interpret performance numbers in context of harness constraints (application sleeps, cluster topology, and load model).